Auditoría completa de software y production readiness

Repositorio AELB · 18 de agosto de 2026

NO LISTO PARA PRODUCCIÓN

Conclusión ejecutiva. La arquitectura base es razonable para una aplicación pequeña, pero existen fallos confirmados de autorización y despliegue que deben impedir la puesta en producción. Los riesgos más graves son la posible exposición de PDFs con datos personales, la escalada de GESTOR a ADMIN, el uso de configuración local/insegura en Docker y la falta de garantías operativas para pagos, migraciones, observabilidad y recuperación.

PRODUCTION READINESS: 42/100

1. Alcance y método

Se inspeccionaron estructura, código Java y TypeScript, configuración, Docker, SQL, migraciones, seguridad, Stripe, frontend, tests, dependencias, documentación y estado del repositorio. No se modificó código de la aplicación. El árbol ya contenía cambios previos en aelb-frontend/app/lib/api.ts, package.json y package-lock.json.

2. Arquitectura detectada

Frontend: Next.js 16.3, React 19, TypeScript.

Backend: Spring Boot 4.0.6, Java 21, Spring Security, JPA/Hibernate.

Base de datos: MySQL 8.4.

Almacenamiento: S3/MinIO con URLs presignadas.

Pagos: Stripe Checkout y webhooks firmados.

Autenticación: JWT en cookies HttpOnly, refresh tokens rotatorios y CSRF.

Roles: USER, GESTOR y ADMIN.

Migraciones: scripts SQL manuales; sin Flyway/Liquibase.

Email, WebSockets y CI/CD: no detectados.

Health checks: MySQL/MinIO en Compose; no backend propio.

3. Bloqueantes para producción

[CRITICAL] C-01 — PDF de inscripciones accesible públicamente

Ubicación: SecurityConfig.java:94-96, RegistrationController.java:198-214.

El matcher protegido cubre /registrations, pero no /registrations/pdf. La ruta cae en GET /api/championships/**.permitAll(). El PDF contiene nombres y DNI.

Solución: añadir antes del matcher público /api/championships/*/registrations/pdf con hasAnyRole("GESTOR","ADMIN") y test anónimo 401.

[CRITICAL] C-02 — Un GESTOR puede convertir usuarios en ADMIN

Ubicación: SecurityConfig.java:145-150, UserController.java:164-180, AdminUpdateUserUseCase.java:18-38.

/api/users/{id}/admin permite GESTOR y ADMIN; el caso de uso acepta role=ADMIN. Esto permite compromiso total de la aplicación.

Solución: solo ADMIN puede cambiar roles; crear endpoint separado para que GESTOR edite únicamente datos no privilegiados.

[CRITICAL] C-03 — Compose de producción activa configuración local

Ubicación: docker-compose.yml:103-122, application.properties:28-53.

Se fuerza el perfil local, cookies Secure por defecto a false, MinIO con credenciales estáticas, URLs HTTP/localhost, puertos publicados e imágenes latest. Desplegar este Compose sin sobrescribir todo produce una instalación insegura o rota.

Solución: separar Compose dev/prod, usar HTTPS, IAM/S3, secretos externos, imágenes fijadas y no publicar MySQL/MinIO.

4. Hallazgos HIGH

[HIGH] H-01 — Migraciones manuales

Ubicación: database/migrations/, documentación de despliegue.

Hibernate valida pero no migra. No hay control de versión aplicado en la base, rollback ni pipeline. Un despliegue puede arrancar con esquema incompleto.

Solución: adoptar Flyway/Liquibase y convertir los scripts en migraciones controladas.

[HIGH] H-02 — Pago confirmado sin inscripción materializada

Ubicación: ConfirmStripeChampionshipBasketPaymentUseCase.java:14-27, RegisterForChampionshipUseCase.java:85-105.

El webhook crea la inscripción después del pago. Si el campeonato cierra o cambia la elegibilidad antes del webhook, Stripe puede haber cobrado y la inscripción ser rechazada.

Solución: reservar/validar la inscripción al crear checkout y materializar esa reserva en el webhook.

[HIGH] H-03 — Webhook de cuota sin comprobar session ID

Ubicación: ConfirmStripeQuotaPaymentUseCase.java:12-18.

Se usa user/año del metadata, pero no se valida contra quota.stripe_session_id. Tampoco hay transacción ni lock explícito.

Solución: localizar por session ID, validar usuario/año/importe y confirmar con lock dentro de transacción.

[HIGH] H-04 — Uploads sin límites suficientes

Ubicación: StorageController.java:25-38 y DTOs de documentos/resultados.

No hay límite de tamaño, validación de magic bytes ni lista blanca estricta de carpetas/tipos. Base64 puede provocar consumo elevado de memoria y almacenamiento.

Solución: límites de API/S3, MIME real, extensiones permitidas y presigned POST con content-length-range.

[HIGH] H-05 — Sin recuperación de contraseña ni emails

No hay password reset, verificación de email ni notificaciones de pago/inscripción. Un usuario que pierda la contraseña queda bloqueado.

[HIGH] H-06 — Errores de Stripe silenciosos

Ubicación: StripeWebhookController.java:49.

Se captura cualquier excepción y se devuelve 400 sin registrar evento, checkout, session, causa ni correlation ID. Un pago fallido queda difícil de diagnosticar.

[HIGH] H-07 — Sin CI/CD

No se detecta pipeline para build, tests, lint, dependencias, imágenes, migraciones ni smoke tests.

[HIGH] H-08 — Frontend no pasa lint

npm run lint devuelve 16 errores y 22 warnings, principalmente por setState síncrono desde useEffect en páginas administrativas, Navbar y Perfil.

5. Hallazgos MEDIUM

M-01: no existe capacidad/aforo en el modelo de campeonato; no se puede impedir superar plazas.

M-02: registros usan read-check-write sin lock; la base protege duplicados exactos, pero no máximos/reglas concurrentes.

M-03: no hay CSP, HSTS ni Referrer-Policy explícitos.

M-04: rate limiter en memoria; no es global con varias réplicas.

M-05: usuarios se cargan completos y se pagina en memoria; existen consultas dentro de streams.

M-06: contenidos y Base64 carecen de límites homogéneos.

M-07: borrado de usuarios puede chocar con foreign keys y no hay política clara de anonimización.

M-08: npm audit --omit=dev detecta vulnerabilidad HIGH transitoria en nanoid, con actualización disponible.

6. Seguridad positiva observada

JWT firmado con secreto mínimo y comprobación de versión de sesión.

Refresh tokens almacenados como hash y rotatorios.

Cookies HttpOnly y CSRF mediante cookie/header.

CORS con lista de orígenes configurada, no wildcard por defecto.

Webhook Stripe con firma y timestamp.

Precio calculado en backend.

Éxito de Stripe no marca el pago por sí mismo.

Checkouts con claves de idempotencia.

Contenido no publicado filtrado en controladores de noticias, campeonatos, resultados y documentos.

Contraseñas con BCrypt y backend Docker no-root.

7. Base de datos, pagos y concurrencia

El esquema tiene claves foráneas, unicidad de usuarios, cuotas y registros, restricciones de importes y checkouts con claves únicas. Hibernate está en validate. Esto es positivo, pero la integridad operativa depende de aplicar correctamente las migraciones.

Las cuotas tienen lock pesimista en el repositorio, pero la confirmación Stripe no lo utiliza. Las inscripciones tienen unicidad por usuario/campeonato/categoría, pero no existe lock para reglas de máximo, turnos o aforo.

Stripe implementa timeout, retry y circuit breaker básicos, pero la confirmación de negocio y la observabilidad todavía no son suficientes para pagos reales.

8. Tests y verificación

TipoEstado

Unitarios de dominioParciales

Seguridad JWT/autorizaciónAlgunos tests

Integración API/JPAMuy limitada

Stripe/webhooksNo detectados

Concurrencia/uploads/migracionesNo detectados

E2E frontendNo existe

Maven no pudo ejecutarse por permisos del target/classes preexistente. El build Next.js quedó bloqueado por una restricción del sandbox al crear procesos de Turbopack; no se clasifica como fallo confirmado de la aplicación.

9. Operación y resiliencia

No hay health/readiness propio del backend.

No hay graceful shutdown explícito.

No hay correlation ID ni logging estructurado suficiente.

No hay alertas para 5xx, inscripciones fallidas o Stripe.

No hay backup/restore automatizado ni restauración probada.

No hay procedimiento formal de rollback.

Se requiere backup de MySQL y objetos S3/MinIO.

10. Tabla resumen

IDSeveridadÁreaProblemaBloquea

C-01CRITICALAutorizaciónPDF de inscripciones fuera del matcher protegidoSí

C-02CRITICALAutorizaciónGESTOR puede promover a ADMINSí

C-03CRITICALDeploymentCompose local, HTTP y defaults insegurosSí

H-01HIGHDBMigraciones manualesSí

H-02HIGHPagosPago Stripe sin inscripción materializadaSí

H-03HIGHPagosWebhook de cuota sin session bindingSí

H-04HIGHUploadsSin límites de tamaño/tipo suficientesSí

H-05HIGHAuthSin password reset/emailsRecomendado

H-06HIGHObservabilidadWebhook silencia excepcionesSí

H-07HIGHCI/CDNo existe pipelineSí

H-08HIGHFrontendLint fallaSí

M-01MEDIUMNegocioNo hay aforo modeladoSegún negocio

M-02MEDIUMConcurrenciaRegistro sin lock completoRiesgo

M-03–M-08MEDIUMOperación/seguridadHeaders, rate limit, paginación, límites, borrado y dependencia npmNo

11. Production readiness score

ÁreaPuntuaciónMotivo principal

Seguridad4/10Fallos críticos de autorización

Fiabilidad4/10Pagos y operación incompletos

Base de datos6/10Buen esquema, migraciones manuales

Arquitectura7/10Separación por dominio razonable

Calidad de código5/10Deuda y lint frontend

Tests2/10Sin API, Stripe, concurrencia ni E2E

Observabilidad2/10Sin health, alertas ni trazabilidad

Configuración3/10Defaults locales en Compose

Deployment3/10Sin CI/CD ni migraciones automatizadas

Mantenibilidad6/10Arquitectura clara, operación incompleta

12. Plan de acción

FASE 1 — Bloqueantes

Corregir SecurityConfig, UserController, AdminUpdateUserUseCase, autorización del PDF y separar Compose dev/prod. Dificultad baja-media. Sin migración salvo política de usuarios. Tests API de roles, ownership y escalada.

FASE 2 — Seguridad

Limitar uploads, MIME, carpetas, tamaño, CSP, HSTS y rate limiting distribuido. Dificultad media. Tests de payloads, CORS, CSRF y headers.

FASE 3 — Fiabilidad

Corregir casos Stripe, session binding, transacciones, reservas e idempotencia. Dificultad alta. Probable migración para eventos Stripe/reservas. Tests de reintentos, retrasos, duplicados y concurrencia.

FASE 4 — Tests

Añadir MockMvc, IDOR/BOLA, publicación, Stripe, migraciones, uploads, errores HTTP, concurrencia y E2E de login/inscripción/pago.

FASE 5 — Observabilidad

Health liveness/readiness, correlation ID, logs estructurados, métricas 5xx/Stripe/inscripciones y alertas.

FASE 6 — Deployment

CI/CD, imágenes fijadas, Flyway/Liquibase, healthchecks, graceful shutdown, backups MySQL/S3, restauración probada y rollback.

FASE 7 — Posterior

Password reset, emails, paginación SQL, anonimización, Redis, optimización N+1, E2E completo y retirada de columnas Base64 legacy.

13. Veredicto final

NO LISTO PARA PRODUCCIÓN

La aplicación puede evolucionar hacia producción sin una reescritura arquitectónica, pero no debe desplegarse hasta resolver C-01, C-02 y C-03, y preferiblemente H-01 a H-08. El riesgo de incidente más probable es una combinación de autorización incorrecta, exposición de datos personales, despliegue con configuración local y un pago Stripe que no se refleje correctamente en la inscripción.